

08 — Menyelesaikan Masalah Nyata dengan Database

Tujuan: Menunjukkan bahwa skema BelajarKUY **bukan teori belaka** — tetapi nyata-nyata **menyelesaikan masalah bisnis** di platform e-learning.

Konteks Masalah

Industri kursus online butuh sistem yang dapat menjawab pertanyaan-pertanyaan ini:

1. Bagaimana cara mengelola **ribuan kursus** dari banyak instruktur tanpa kekacauan data?
2. Bagaimana memastikan **pembayaran tidak ganda** atau **hilang** di tengah transaksi?
3. Bagaimana memberi user **akses ke kursus** secara otomatis setelah bayar?
4. Bagaimana **mencegah review fake** dari user yang tidak pernah beli?
5. Bagaimana melacak **progres belajar** per user per lecture?
6. Bagaimana **menghitung pendapatan** instruktur untuk bagi-hasil?

Setiap masalah di atas **diselesaikan oleh keputusan desain database** kita.

Masalah 1 — Mencegah Pembelian Duplikat

Skenario

User klik tombol "Beli Sekarang" 2× karena koneksi lambat. Tanpa pencegahan, dia akan dibuatkan **2 enrollment** untuk kursus yang sama → bayar 2× untuk barang yang sama.

Solusi Database

UNIQUE constraint pada `enrollments(user_id, course_id)`:

```
CREATE TABLE enrollments (  
    ...  
    UNIQUE KEY enrollments_user_course_unique (user_id, course_id)  
);
```

Akibatnya, percobaan insert kedua akan **otomatis ditolak** oleh database dengan error:

```
Duplicate entry '3-7' for key 'enrollments_user_course_unique'
```

Sistem tangkap error itu, lalu kembalikan response "Anda sudah membeli kursus ini" — **tanpa data ganda masuk database**.

Masalah 2 — Pembayaran Atomic (All-or-Nothing)

? Skenario

User bayar 3 kursus sekaligus seharga Rp 600.000. Saat sistem ingin buat 3 baris di **orders**, baris ke-2 gagal karena server overload. Tanpa atomicity → user kena charge Rp 600.000 tapi cuma dapat akses 1 kursus.

✅ Solusi Database

TRANSACTION + relasi **payments** → **orders**:

```
START TRANSACTION;

INSERT INTO payments (...) VALUES (...);
SET @pid = LAST_INSERT_ID();

INSERT INTO orders (payment_id, ...) VALUES (@pid, ...);
INSERT INTO orders (payment_id, ...) VALUES (@pid, ...);
INSERT INTO orders (payment_id, ...) VALUES (@pid, ...);

-- Jika SEMUA sukses:
COMMIT;

-- Jika ada yang gagal:
ROLLBACK;
```

Database **menjamin** kalau salah satu gagal → semua dibatalkan, atau semua tercatat. **Tidak ada state setengah jadi**. Ini fitur engine InnoDB (ACID).

📌 Masalah 3 — Otomasi Pemberian Akses

? Skenario

Setelah Midtrans webhook konfirmasi pembayaran sukses, user harus **otomatis** dapat akses ke kursus tanpa intervensi admin.

✅ Solusi Database

Saat status **payments.status** berubah dari **pending** → **settlement**, application logic membuat enrollment dari setiap order:

```
INSERT INTO enrollments (user_id, course_id, order_id, enrolled_at)
SELECT
  o.user_id,
  o.course_id,
  o.id,
  NOW()
FROM orders o
WHERE o.payment_id = :payment_id
AND o.status      = 'completed';
```

Karena ada UNIQUE pada (`user_id`, `course_id`), query ini **idempotent** — bisa dijalankan berkali-kali tanpa risiko data ganda (kalau webhook Midtrans terkirim 2×, tetap aman).

📌 Masalah 4 — Mencegah Review Palsu

? Skenario

Kompetitor bikin akun palsu, kasih bintang 1 ke kursus saingan padahal belum pernah beli. Atau user pengen kasih review tapi tidak benar-benar pernah beli.

✅ Solusi Database

Pada saat menyimpan review, tambahkan **EXISTS check** ke `enrollments`:

```
INSERT INTO reviews (user_id, course_id, rating, comment, status, created_at,
updated_at)
SELECT :user_id, :course_id, :rating, :comment, 'pending', NOW(), NOW()
WHERE EXISTS (
  SELECT 1 FROM enrollments
  WHERE user_id = :user_id AND course_id = :course_id
);
```

Kalau user **tidak punya enrollment** → tidak ada baris yang di-insert → review otomatis tidak terbentuk.

Bonus: UNIQUE pada `reviews(user_id, course_id)` mencegah 1 user spam review berkali-kali pada kursus yang sama.

📌 Masalah 5 — Tracking Progres Belajar Real-time

? Skenario

User mau lihat dashboard "saya sudah selesai 8 dari 30 video di kursus ini (27%)". Server tidak boleh hitung ulang seluruh data setiap user buka dashboard.

✅ Solusi Database

Tabel `lecture_completions` mencatat granular per lecture:

```
-- Saat user finish 1 video
INSERT IGNORE INTO lecture_completions (user_id, lecture_id, completed_at)
VALUES (:user_id, :lecture_id, NOW());

-- Hitung progres per kursus (cached via query)
SELECT
  c.id,
  c.title,
  COUNT(DISTINCT cl.id) AS total_video,
  COUNT(DISTINCT lc.lecture_id) AS sudah_ditonton,
  ROUND(
    COUNT(DISTINCT lc.lecture_id) * 100.0 /
    NULLIF(COUNT(DISTINCT cl.id), 0),
    0) AS persen
FROM enrollments e
JOIN courses      c ON c.id          = e.course_id
JOIN course_sections cs ON cs.course_id = c.id
JOIN course_lectures cl ON cl.section_id = cs.id
LEFT JOIN lecture_completions lc
  ON lc.lecture_id = cl.id
  AND lc.user_id   = e.user_id
WHERE e.user_id = :user_id
GROUP BY c.id, c.title;
```

Query ini cepat karena pakai index pada `(user_id, lecture_id)`.

🔪 Masalah 6 — Bagi Hasil Instruktur

? Skenario

Setiap bulan, platform harus transfer 70% dari penjualan ke setiap instruktur. Bagaimana hitungnya tanpa salah?

✅ Solusi Database

Karena setiap `orders` punya `instructor_id`, hitung pendapatan per instruktur sangat mudah:

```
SELECT
  u.id,
  u.name AS instruktur,
  SUM(o.final_price) AS total_penjualan,
  ROUND(SUM(o.final_price) * 0.7) AS bagi_hasil_instruktur,
  ROUND(SUM(o.final_price) * 0.3) AS komisi_platform
FROM users u
JOIN orders o ON o.instructor_id = u.id
WHERE o.status = 'completed'
  AND MONTH(o.created_at) = MONTH(CURDATE())
```

```
AND YEAR(o.created_at) = YEAR(CURDATE())
GROUP BY u.id, u.name
ORDER BY total_penjualan DESC;
```

Karena **original_price**, **discount_amount**, dan **final_price** **disnapshot** di order, perubahan harga kursus tidak mengganggu laporan bagi-hasil bulan lalu.

📌 Masalah 7 — Validasi Kupon yang Benar

? Skenario

User pakai kupon "DISKON50". Sistem harus pastikan:

- Kupon **ada & aktif**
- Belum **kadaluarsa**
- Belum **habis terpakai**
- Berlaku untuk **kursus yang dibeli**

✅ Solusi Database

```
SELECT id, discount_percent
FROM coupons
WHERE code      = :input_code
  AND status    = 1
  AND valid_until >= CURDATE()
  AND (max_usage IS NULL OR used_count < max_usage)
  AND (course_id IS NULL OR course_id = :course_id)
LIMIT 1;
```

Kalau query return 0 baris → kupon **tidak valid**. Aman & deklaratif.

Setelah kupon dipakai, tinggal update counter:

```
UPDATE coupons
SET used_count = used_count + 1
WHERE id = :coupon_id;
```

📌 Masalah 8 — Skalabilitas dengan Index

? Skenario

Setelah 100,000 kursus, query homepage "kursus featured + active" jadi lambat karena scan seluruh tabel.

✅ Solusi Database

Composite index:

```
CREATE INDEX courses_status_featured_index ON courses (status, featured);
```

Setelah index dibuat:

```
SELECT id, title, price, thumbnail
FROM courses
WHERE status = 'active' AND featured = 1
ORDER BY created_at DESC
LIMIT 12;
```

Query berubah dari **full table scan** → **index range scan**. Speed-up: dari ~2 detik → ~10 milidetik (untuk 100k row).

Masalah 9 — Audit Trail Pembayaran

Skenario

User komplain "saya sudah bayar tapi gak dapat akses". Customer support butuh data lengkap: berapa yang dibayar, kapan, lewat channel apa, respon Midtrans seperti apa.

Solusi Database

Tabel **payments** menyimpan **JSON respon Midtrans utuh**:

```
CREATE TABLE payments (
  ...
  midtrans_response JSON NULL,
  ...
);
```

```
-- Trace pembayaran untuk audit
SELECT
  midtrans_order_id,
  status,
  payment_type,
  total_amount,
  JSON_EXTRACT(midtrans_response, '$.transaction_status') AS status_midtrans,
  JSON_EXTRACT(midtrans_response, '$.fraud_status') AS fraud_status,
  created_at
FROM payments
WHERE user_id = :user_id
ORDER BY created_at DESC;
```

Tidak ada lagi pertanyaan "ini bayarnya gimana ya?" — semua jejak ada di database.

🔗 Masalah 10 — Soft Delete / Akses Berlanjut

? Skenario

Instruktur memutuskan menghapus kursus lama. Tapi 50 user sudah enroll. Apakah akses mereka harus hilang?

✅ Pilihan Desain

BelajarKUY memilih: **ketika instruktur dihapus** → kursus juga dihapus (CASCADE). Logikanya, kursus tanpa instruktur tidak bisa di-support (tidak ada Q&A, tidak ada update materi).

Alternatif yang **tidak dipilih** tapi mungkin:

- **Soft delete:** tambah kolom `deleted_at`, kursus tidak benar-benar hilang
- **Transfer ownership:** ganti `instructor_id` ke admin sebelum hapus user

Keputusan ini **didokumentasikan** sehingga developer baru tahu konsekuensinya.

🎯 Kesimpulan: Database = Tulang Punggung Bisnis

Database BelajarKUY **bukan sekedar tempat menyimpan data**, tapi:

Fungsi	Mekanisme
🛡️ Penjaga Integritas	FK constraint, UNIQUE, NOT NULL
⚡ Akselerator Query	Composite index pada kolom hot
🔒 Penjamin Atomicity	TRANSACTION + InnoDB
📄 Audit Trail	created_at, updated_at, JSON response Midtrans
💰 Sumber Kebenaran Finansial	Snapshot harga di orders

Fungsi

Mekanisme



Logika Bisnis Deklaratif

Cascade policy, default value, ENUM state

Tanpa keputusan-keputusan desain di atas, BelajarKUY tidak akan bisa beroperasi sebagai marketplace yang **aman, cepat, dan terpercaya**.



Take-Home untuk Audience

"Database yang **dirancang dengan baik** = setengah aplikasi sudah jadi. Sisanya tinggal menerjemahkan request user menjadi query SQL yang tepat — karena **business rule sudah dikodekan ke dalam schema**."



Terima kasih! Pertanyaan?